

SMART CAMPUS EVENT MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

M A S THULASI NARAYANA REDDY (VTU26657)

10211CS224

Full Stack Application Development

Slot : E1-B2



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

MAY, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "Smart Campus Event Management System " by M A S THULASI NARAYANA REDDY (VTU26657) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.S.Hemamalini

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

MAY, 2026

Signature of Course Coordinator

Dr. M.Sumithra

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

MAY, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(M A S THULASI NARAYANA REDDY)

Date:06/05/2026

ABSTRACT

The Smart Event Management System is a web-based application developed to enhance the efficiency and reliability of organizing and managing events within institutions and organizations. Conventional event management practices often rely on manual coordination, leading to challenges such as scheduling conflicts, inaccurate participant tracking, inefficient communication, and limited real-time visibility of event status. This project introduces a digital solution that leverages modern web technologies to streamline the entire event lifecycle.

The system is built using a React-based architecture, utilizing component-driven design and React Hooks for effective state management and dynamic user interaction. It provides a centralized platform where organizers can create, update, and manage events, while participants can easily browse event details, register, and receive instant confirmations. Key features include automated registration handling, real-time seat availability tracking, validation mechanisms to prevent overbooking, and interactive dashboards for monitoring event participation.

Keywords:

Mysql, Spring MVC, Spring MVC, Spring Data JPA, Spring Data JPA, Thymeleaf, Event Registration, Admin Dashboard, SDG 9 .

LIST OF FIGURES

1.1	Dashboard	3
3.1	Backend Architecture Diagram	8
4.1	Data Flow Diagram	12
5.1	Admin page	16
5.2	Registration page	18

LIST OF TABLES

2.1	Comparison of Campus Event Management Platforms	5
5.1	Comparative Analysis of Event Management Methods	17
7.1	Mapping of Project to Complex Engineering Problem	21
7.2	Mapping of Project to Complex Engineering Activities (CEA) . . .	22
7.3	SDG Mapping for the Project	23

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DI	Dependency Injection
FSAD	Full Stack Application Development
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JPA	Java Persistence API
JVM	Java Virtual Machine
MVC	Model View Controller
REST	Representational State Transfer
SDG	Sustainable Development Goal
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	2
1.4 Framework	2
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
2.1 Market Leaders and Competitors	4
3 FRAMEWORK DESCRIPTION	6
3.1 Frontend Implementation	6
3.2 Backend Implementation	7
3.3 Database Implementation	8

4	METHODOLOGIES	11
4.1	Project Architecture	11
4.2	DataFlow Diagram	12
4.3	Module 1: Event Management Module	12
4.4	Module 2: Registration Module	13
4.5	Module 3: Admin Dashboard Module	13
4.6	Module 4: Confirmation and Feedback Module	14
5	RESULTS AND DISCUSSIONS	16
6	CONCLUSION AND FUTURE ENHANCEMENTS	19
6.1	Conclusion	19
6.2	Future Enhancements	20
7	Addressing Complex Engineering Problem and SDG	21
7.1	Mapping of the Project to Complex Engineering Problem (CEP) . .	21
7.2	Mapping of the Project to Complex Engineering Activities (CEA) .	22
7.3	SDG Mapping (CEP Section)	23
8	SOURCE CODE	24
8.1	Source Code	24
	References	28

Chapter 1

INTRODUCTION

1.1 Introduction

Traditional event management in colleges is often handled manually, leading to inefficiencies such as scheduling conflicts, lack of centralized data, and difficulty in tracking registrations. These manual processes increase the chances of data inconsistency, delayed communication, and limited visibility of event participation. This project proposes a Smart Campus Event Management System to digitize and streamline event-related activities. The system provides a centralized platform where students can view and register for events, while administrators can efficiently manage event data. By integrating a structured backend with a dynamic frontend, the system ensures accurate data handling, real-time updates, and improved user experience.

1.2 Aim of the Project

The primary aim of this project is to develop a full stack web application that simplifies campus event management through automation and centralized control. It focuses on enabling students to easily browse and register for events while allowing administrators to manage event details efficiently. The system aims to reduce manual errors, improve data accuracy, and provide real-time tracking of registrations.

1.3 Scope of the Project

The scope of this project includes the design and implementation of a web-based system with the following functionalities:

Event Management: Creation, updating, and deletion of events with details such as event name, date, department, and description.

User Registration: Allowing students to register for events through validated forms and view their registered events.

Admin Control: Providing administrators with tools to manage events, search using filters, and monitor participation.

System Functionality: Ensuring real-time data updates, secure access, and efficient handling of user requests.

Validation: Implementing input validation to ensure data accuracy and prevent invalid submissions.

1.4 Framework

The project is developed using a full stack framework based on Spring technologies and follows the MVC architecture for better modularity and scalability. The core technical framework includes:

Backend: Spring Boot for application development with embedded server support and REST API creation.

Web Layer: Spring MVC for handling requests using annotation-based controllers.

Database Layer: Spring Data JPA for entity mapping, database interaction, and CRUD operations.

Frontend: Thymeleaf templates integrated with HTML5 and CSS3 for dynamic content rendering.

Validation and Security: Bean validation annotations and basic authentication for secure admin access.

Development Environment: Visual Studio Code (VS Code) or Eclipse IDE for development, and Git/GitHub for version control.

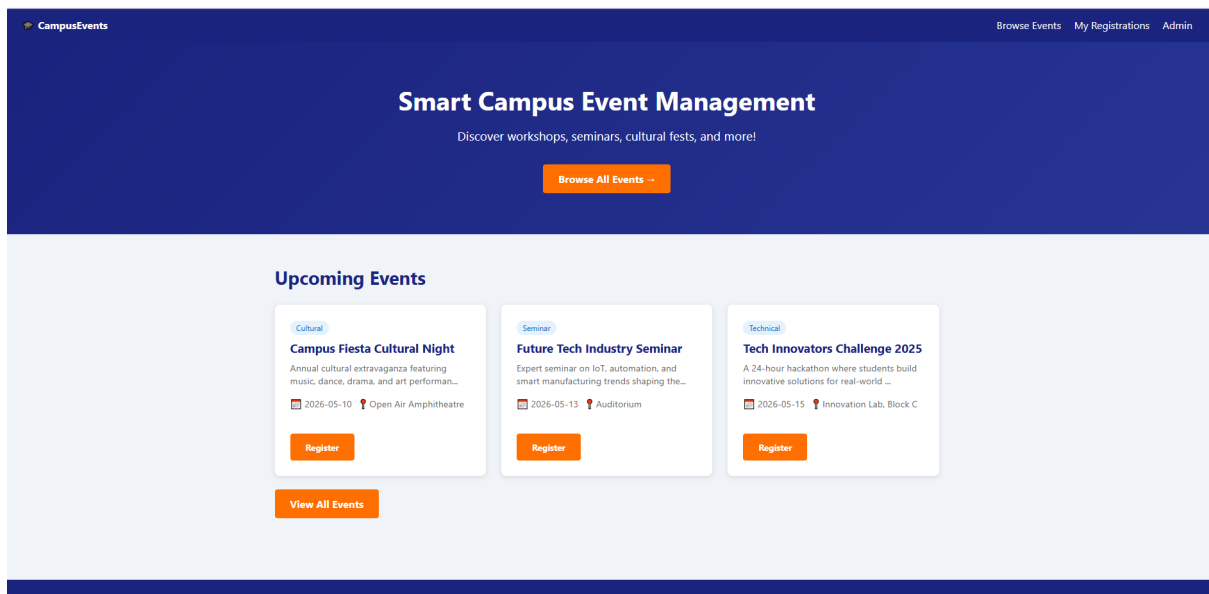


Figure 1.1: Dashboard

Figure 1.1 shows the dashboard of the Smart Campus Event Management System. It provides a centralized interface for users to view and access system features.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

The campus event management landscape in 2026 is increasingly focused on digital transformation, moving from manual coordination to automated, centralized systems. Modern platforms emphasize real-time tracking, seamless registration, and efficient event coordination to enhance user experience within educational institutions.

2.1 Market Leaders and Competitors

Existing solutions are often either too expensive or overly complex for small to medium-sized academic institutions. The proposed Smart Campus Event Management System provides a simplified, cost-effective solution tailored for campus use. It focuses on efficient event creation, student registration, and administrative control, ensuring improved usability and real-time data management within the institution.

Table 2.1: Comparison of Campus Event Management Platforms

Platform	Primary Use Case	Key Strengths	Main Limitations
Eventbrite	Public and institutional event registration	Easy event setup; supports online registration and ticketing	Transaction fees; not fully customizable for campus needs
Zoho Backstage	Corporate and academic event management	Integrated event tools; RSVP tracking and analytics	Advanced features require paid plans
Cvent	Large-scale conferences and university events	Advanced analytics; robust event management features	High cost; complex setup for small institutions
Campus Labs	University-specific event and student engagement	Tailored for campus use; supports student involvement tracking	Limited flexibility outside academic environments

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend environment for the Smart Campus Event Management System was established using standard web technologies including **HTML5**, **CSS3**, and **Thymeleaf**. Thymeleaf is integrated with Spring Boot to dynamically render data on web pages. **Visual Studio Code (VS Code)** or Eclipse IDE was used as the development environment for writing and debugging the frontend code. **Git** and GitHub were configured for version control to track project changes and enable collaborative development.

3.1.2 Structure of the Project

The frontend follows a template-based architecture using Thymeleaf, enabling seamless integration with backend data. The project structure includes:

- **Templates Folder:** Contains Thymeleaf HTML pages such as event listing, registration form, and admin dashboard.
- **Static Folder:** Stores CSS, JavaScript, and image files for styling and interactivity.

- **Main Template Pages:** Includes reusable layouts for headers, footers, and navigation components.

3.1.3 Execution Procedure

The frontend is executed as part of the Spring Boot application. When the server is started, the application can be accessed through a **web browser** such as Chrome or Edge. Users can view available events, register through validated forms, and view confirmation messages. The system ensures proper **input validation** for fields such as name, email, and event details before submission.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend of the Smart Campus Event Management System is developed using **Spring Boot** with **Java Development Kit (JDK)**. The embedded **Tomcat server** provided by Spring Boot is used to run the application. **MySQL** is used as the relational database for storing event and user data. Development tools include Spring Boot dependencies for web, JPA, and validation to support full stack application development.

3.2.2 Structure of the Project

The backend follows the **MVC (Model-View-Controller)** architecture to ensure modularity and maintainability. It consists of:

- **Controller Layer:** Handles HTTP requests using annotations such as `@Controller` and `@RequestMapping`, and interacts with the service layer.
- **Service Layer:** Contains business logic such as event creation, registration processing, and validation.

- **Repository Layer:** Uses Spring Data JPA for database operations like saving, updating, and retrieving event data.
- **Model Layer:** Defines entities such as Event and User using annotations like `@Entity`.

3.2.3 Execution Procedure

The backend is executed by running the Spring Boot application, which starts the embedded server. Once the application is active, it listens for incoming requests from the frontend. When a user registers for an event, the system processes the request, validates the input data, and stores it in the database. The system then updates event participation details and returns a response to display confirmation to the user. Administrative operations such as adding, editing, or deleting events are handled securely through authenticated access.

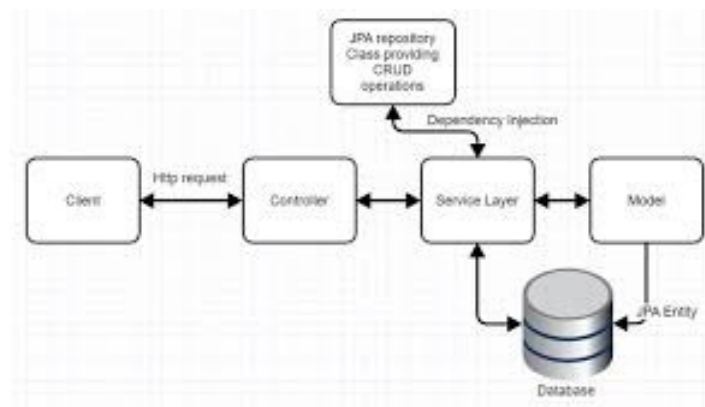


Figure 3.1: Backend Architecture Diagram

Figure 3.1 shows the backend architecture of the Smart Campus Event Management System. It illustrates the interaction between different layers such as controller, service, repository, and database.

3.3 Database Implementation

3.3.1 Database Configuration

The database for the **Smart Campus Event Management System** is configured to ensure efficient data storage, retrieval, and integrity for event management and student registrations. A relational database management system such as **MySQL** is used to store persistent data related to events, users, and registrations. The database is integrated with the backend using **Spring Data JPA**, which simplifies database interactions through object-relational mapping.

Connection properties such as database URL, username, and password are defined in the `application.properties` file. This configuration allows the Spring Boot application to establish a secure connection and perform CRUD operations seamlessly.

3.3.2 Schema Structure

The database schema is designed using a **relational model** to maintain structured and consistent data. It follows a modular approach where entities are logically separated and connected through relationships. The schema supports core functionalities such as event creation, student registration, and tracking participation.

Relationships between tables ensure data integrity, for example linking event records with user registrations using foreign keys. This structure enables efficient querying, filtering, and aggregation of data for administrative purposes such as viewing event statistics and participation details.

3.3.3 Tables Created

The following primary tables are implemented to manage the system data:

- **Events Table:** Stores details of all campus events, including a unique `EventID`, Event Name, Date, Department, Description, and Venue.
- **Registrations Table:** Maintains records of student participation, including a unique `RegistrationID`, Student Name, Email ID, associated `EventID`, and registration timestamp.

- **Users Table:** Stores login credentials and roles (Admin/Student), including User ID, Username/Email, and Password for authentication and authorization.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The system follows a **Model-View-Controller (MVC) Architecture** implemented using the **Spring Boot** framework to ensure modularity, scalability, and maintainability. The architecture is divided into three primary layers:

- **Controller Layer:** Handles incoming HTTP requests from users. It processes actions such as viewing events, registering for events, and admin operations like adding or updating event details.
- **Service Layer:** Contains the core business logic of the application, including event management, validation of user inputs, and processing of event registrations.
- **Repository Layer:** Uses **Spring Data JPA** to interact with the database. It performs CRUD operations for entities such as events, users, and registrations.
- **View Layer:** Developed using **Thymeleaf**, it dynamically renders data on web pages and provides an interactive interface for both students and administrators.

Data Flow: The process begins when a user interacts with the frontend interface (e.g., registering for an event). The request is sent to the controller, which forwards

it to the service layer for processing. The service layer performs validation and business logic, then communicates with the repository layer to store or retrieve data. Finally, the response is returned to the view layer, where updated information is displayed to the user.

4.2 DataFlow Diagram

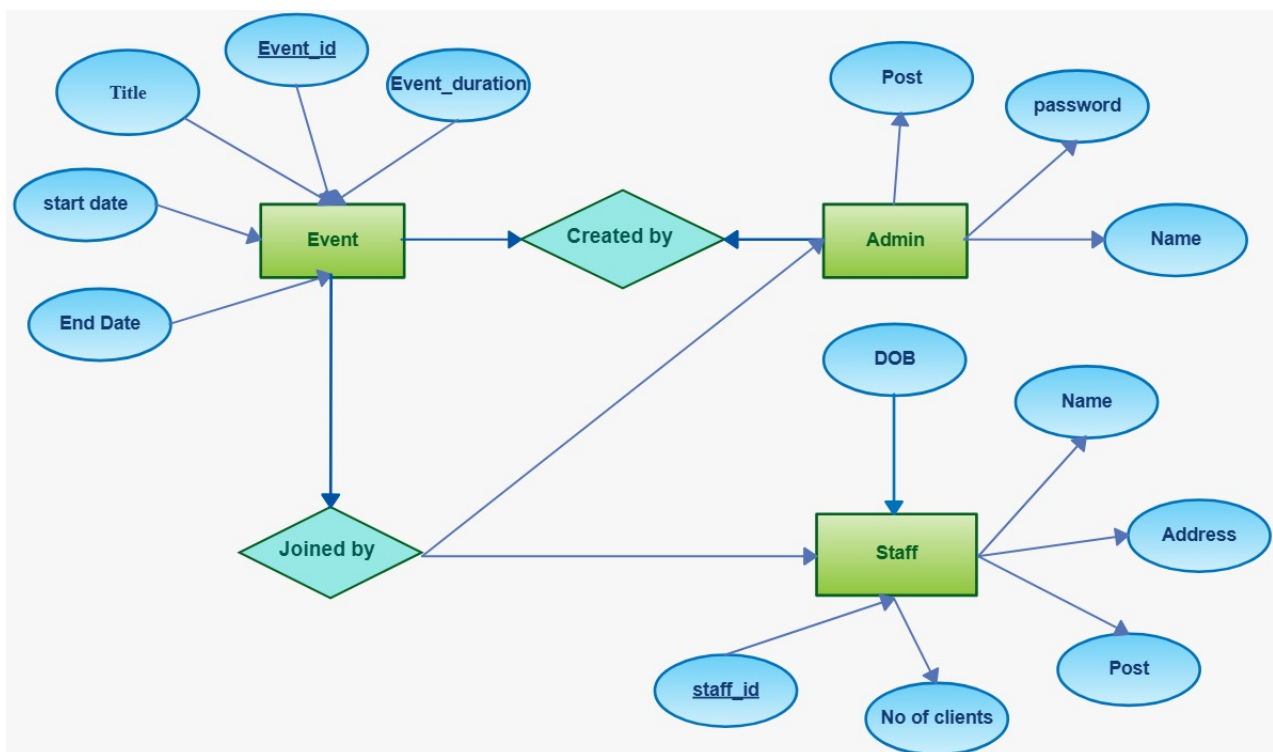


Figure 4.1: Data Flow Diagram

Figure 4.1 shows the Data Flow Diagram (DFD) of the Smart Campus Event Management System. It illustrates how data flows between users, administrators, system processes, and the database.

4.3 Module 1: Event Management Module

The **Event Management Module** acts as the core administrative component of the system:

- **Event Creation:** Allows administrators to add new events with details such as event name, department, date, time, venue, and description.
- **Event Display:** Displays all available events to users in a structured format using dynamic web pages.
- **Event Modification:** Enables updating and deletion of existing events to maintain accurate and up-to-date information.

4.4 Module 2: Registration Module

This module handles user interaction and registration processes:

- **User Input:** Allows students to enter details such as name, email, and department to register for events.
- **Validation Logic:** Ensures that all required fields are filled and validates inputs using annotations such as `@NotNull` and `@Email`.
- **Registration Handling:** Stores user registration details in the database and associates them with the selected event.

4.5 Module 3: Admin Dashboard Module

The **Admin Dashboard Module** provides administrative control and monitoring features:

- **Event Management Control:** Enables administrators to add, edit, and delete events through a secured interface.
- **Search and Filters:** Allows filtering of events based on date, department, or event type.

- **Statistics and Reports:** Displays registration statistics and participation details using aggregated data.

4.6 Module 4: Confirmation and Feedback Module

This module provides feedback and confirmation to users:

- **Registration Confirmation:** Displays a success message after a user registers for an event.
- **User Feedback:** Allows participants to provide feedback on events for future improvements.

Advantages of this Project

The system offers several key benefits over traditional manual methods:

1. **Efficiency:** Automates event management and registration processes, reducing manual errors.
2. **User Experience:** Provides a simple and interactive interface for both students and administrators.
3. **Scalability:** The modular MVC architecture allows easy expansion and maintenance.
4. **Data Integrity:** Ensures accurate data handling through validation and database integration.
5. **Centralized Management:** Offers a single platform for managing all campus events.

Disadvantages

Despite its advantages, the system has certain limitations:

1. **Internet Dependency:** Requires a stable internet connection for access and operation.
2. **Limited Advanced Features:** Advanced functionalities such as real-time notifications or mobile integration may not be included.
3. **Basic Security Implementation:** Uses basic authentication; advanced security mechanisms can be enhanced in future versions.
4. **Scalability Constraints:** May require optimization when handling a very large number of users or events.
5. **No Payment Integration:** The system does not include online payment features, limiting it to free campus events.

Chapter 5

RESULTS AND DISCUSSIONS

Implementation screenshots

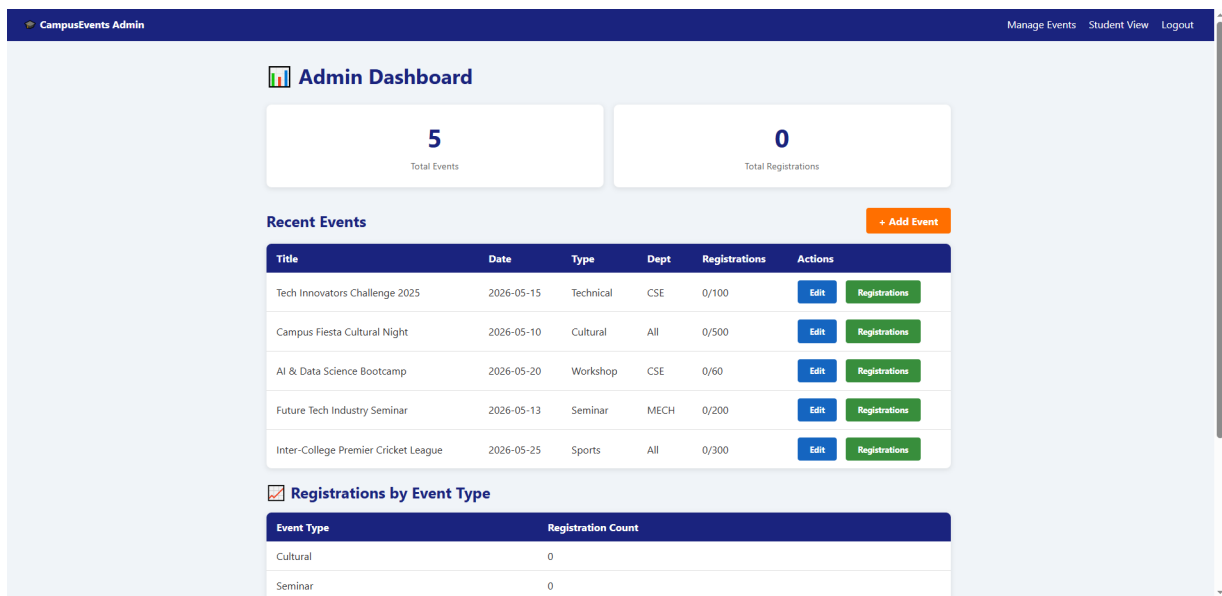


Figure 5.1: Admin page

Figure 5.1 shows the admin page of the Smart Campus Event Management System. This page is designed for administrators to efficiently manage and control all activities related to campus events.

Table 5.1: Comparative Analysis of Event Management Methods

Feature / System	Manual / Traditional System	Smart Campus Event Management System
Registration Speed	Slow (Manual entry and approvals)	Fast (Online registration system)
Data Accuracy	Low (Prone to human errors)	High (Validated input and database storage)
Tracking	Paper-based or spreadsheets	Real-time tracking through database
Event Management	Manual coordination	Centralized admin dashboard
User Confirmation	Verbal or manual acknowledgment	Instant confirmation on submission
Availability Information	Requires manual inquiry	Easily accessible event details online
Data Storage	Physical records	Secure digital database (MySQL)

CampusEvents

Browse Events My Registrations

Register for Event

Campus Fiesta Cultural Night
2026-05-10 Open Air Amphitheatre

Full Name *
Enter your full name

Email *
student@college.edu

Roll Number *
e.g. 21CS001

Department *
-- Select --

☒ Register

Figure 5.2: Registration page

Figure 5.2 shows the registration page of the Smart Campus Event Management System. This page is designed to allow new users to create an account and gain access to the platform's features.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The **Smart Campus Event Management System** successfully demonstrates a digital solution for managing campus events by replacing inefficient manual processes. Through the implementation of **Spring Boot**, **Spring MVC**, and **Thymeleaf**, the application provides a structured and user-friendly interface that ensures efficient event management and registration.

The system effectively utilizes **Spring Data JPA** for database operations, enabling real-time data handling and maintaining data integrity. Features such as event creation, student registration, and administrative control streamline the overall workflow and reduce manual errors. Input validation mechanisms ensure accurate data entry, while the centralized system improves accessibility and monitoring of events.

By automating event management tasks and providing a scalable architecture, the project enhances user experience for both students and administrators. This system achieves its intended objectives and contributes to **SDG 9** by promoting innovation and strengthening digital infrastructure within educational institutions.

6.2 Future Enhancements

The current version of the system provides a strong foundation for campus event management. However, several enhancements can be implemented to improve its functionality and scalability:

- **Advanced Authentication:** Implementing role-based access control and multi-factor authentication to enhance system security.
- **Mobile Application:** Developing a mobile-friendly or cross-platform application to improve accessibility for students and faculty.
- **Analytics Dashboard:** Adding advanced reporting and visualization tools to provide insights into event participation and trends.
- **Payment Integration:** Incorporating secure payment gateways for handling paid events within the system.

Existing Applications and Review Comments

Several existing platforms provide useful benchmarks for future enhancements:

- **Eventbrite:** (<https://www.eventbrite.com>) Known for its ease of use and wide event discovery features, but includes service charges that may not suit institutional use.
- **Zoho Backstage:** (<https://www.zoho.com/backstage/>) Provides comprehensive event management tools and analytics, though it may be complex for smaller campus-level applications.
- **Cvent:** (<https://www.cvent.com>) Offers enterprise-level event solutions with strong analytics but is expensive and resource-intensive.

Chapter 7

Addressing Complex Engineering Problem and SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 7.1: Mapping of Project to Complex Engineering Problem

Level	Attribute	Characteristics	Wks	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK4, WK5	Knowledge of React JS, hooks, validation, UI design
WP2	Conflicting Requirements	Technical and non-technical constraints	WK4, WK5	Balances usability with booking constraints and validation
WP3	Depth of Analysis	Analytical and design thinking required	WK3, WK4	Component design, state handling, dynamic updates
WP4	Familiarity	Partially new problems	WK4	Real-time booking and ticket updates
WP5	Applicable Codes	Limited availability of standard solutions	WK6	Custom booking logic and validation logic
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6	Students, faculty, organizers interaction
WP7	Interdependence	Integration of multiple subsystems	WK3, WK5	UI, state, validation integration

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	Utilizes Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML/CSS, and relational databases for full-stack development.
EA2	Level of Interactions	Complex interactions between modules	Manages interactions between event management, user registration, validation, authentication, and reporting modules.
EA3	Innovation	Application of modern technologies	Implements a web-based centralized platform with filtering, real-time updates, and secure admin operations.
EA4	Consequences to Society and Environment	Impact on users and society	Enhances student engagement and improves institutional efficiency while reducing manual errors and delays.
EA5	Familiarity	Beyond standard practices	Combines web development, security, and data analytics into a unified system, extending beyond traditional event handling methods.

7.3 SDG Mapping (CEP Section)

Table 7.3: SDG Mapping for the Project

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Facilitates student participation in academic events, workshops, and seminars, enhancing learning beyond the classroom.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in educational institutions through a centralized event management system.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives by digitizing event processes and improving resource management.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure and transparent event handling through authentication and controlled access.
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling online event registration and management.

Chapter 8

SOURCE CODE

8.1 Source Code

```
package com.example.event;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class EventManagementApplication {
    public static void main(String[] args) {
        SpringApplication.run(EventManagementApplication.class, args);
    }
}

package com.example.event.model;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity

public class Event {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
```

```

private String description;
private LocalDate eventDate;
private String location;
private int capacity;

public Event() {}

public Event(String title , String description , LocalDate eventDate , String location , i
    this.title = title;
    this.description = description;
    this.eventDate = eventDate;
    this.location = location;
    this.capacity = capacity;
}

// Getters and Setters
public Long getId() { return id; }
public String getTitle() { return title; }
public void setTitle(String title) { this.title = title; }

public String getDescription() { return description; }
public void setDescription(String description) { this.description = description; }

public LocalDate getEventDate() { return eventDate; }
public void setEventDate(LocalDate eventDate) { this.eventDate = eventDate; }

public String getLocation() { return location; }
public void setLocation(String location) { this.location = location; }

public int getCapacity() { return capacity; }
public void setCapacity(int capacity) { this.capacity = capacity; }
}

package com.example.event.model;

import jakarta.persistence.*;

@Entity
public class User {

```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

private String name;
private String email;

public User() {}

public User(String name, String email) {
    this.name = name;
    this.email = email;
}

public Long getId() { return id; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }

public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
}

package com.example.event.model;

import jakarta.persistence.*;

@Entity
public class Registration {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    private Event event;

    @ManyToOne
    private User user;

    public Registration() {}

```

```

        public Registration(Event event, User user) {
            this.event = event;
            this.user = user;
        }
    }
}

package com.example.event.repository;

import com.example.event.model.Event;
import org.springframework.data.jpa.repository.JpaRepository;

public interface EventRepository extends JpaRepository<Event, Long> {}
package com.example.event.repository;

import com.example.event.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {}
package com.example.event.repository;

import com.example.event.model.Registration;
import org.springframework.data.jpa.repository.JpaRepository;

public interface RegistrationRepository extends JpaRepository<Registration, Long> {}
package com.example.event.controller;

import com.example.event.model.*;
import com.example.event.repository.*;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/register")
public class RegistrationController {

    private final RegistrationRepository regRepo;
    private final EventRepository eventRepo;
    private final UserRepository userRepo;

    public RegistrationController(RegistrationRepository regRepo,

```

```

        EventRepository eventRepo,
        UserRepository userRepo) {
    this.regRepo = regRepo;
    this.eventRepo = eventRepo;
    this.userRepo = userRepo;
}

@PostMapping("/{eventId}/{userId}")
public String register(@PathVariable Long eventId, @PathVariable Long userId) {

    Event event = eventRepo.findById(eventId).orElseThrow();
    User user = userRepo.findById(userId).orElseThrow();

    regRepo.save(new Registration(event, user));
    return "Registered■Successfully";
}
}

```

References

- [1] G. Akshay Kumar, B. Avinash, K. Meghana, and M. Thapasya, “Smart Campus Event Management and Recommendation System,” *International Journal of Scientific Research in Engineering and Management*, vol. 10, no. 4, pp. 1–9, Apr. 2026.

This paper proposes a centralized smart platform with scheduling, notifications, and personalized event recommendations to improve campus event participation.

- [2] A. Pansare, A. Patil, N. Patil, Y. Patil, and A. Bhonde, “Smart College Event Management System Using MERN Stack,” *International Journal for Research in Applied Science and Engineering Technology*, vol. 11, no. 3, pp. 2125–2129, Mar. 2023.

Describes a web-based system that automates event handling, improves data management, and reduces manual effort in colleges.

- [3] A. Chaturvedi, K. Sharma, A. Dua, and A. Gupta, “CU-Events: A Comprehensive Event Management System for University,” *International Journal for Research in Applied Science and Engineering Technology*, vol. 12, no. 11, pp. 223–229, Nov. 2024.

Introduces a university-level event system with real-time tracking, centralized management, and improved user engagement.

- [4] V. Balu, P. Vagish Krishnan, and M. Varadarajan, “University Event Management System,” *International Journal of Science and Research*, vol. 10, no. 4, pp. 965–966, Apr. 2021.

Focuses on replacing traditional spreadsheets with automated systems for efficient event data handling and report generation.

- [5] S. K. Patel and D. Mehta, “Automated Scheduling and Conflict Detection in Resource Management Systems,” *Procedia Computer Science*, 2021.

Explains scheduling algorithms and conflict detection useful for smart campus event planning.

- [6] P. Singh and M. Verma, “Recommendation Systems in Academic Event Platforms,” *Applied Computing and Informatics*, 2022.

Discusses recommendation techniques to personalize event suggestions for users.

- [7] A. Sharma et al., “Smart Campus Applications Using IoT and Web Technologies,” *IEEE Access*, vol. 11, 2023.

Highlights IoT-based smart campus systems that enhance automation and real-time communication.